FH JOANNEUM - University of Applied Sciences

Dynamic Resource Allocation and Scaling of Stateful Workloads in Kubernetes

A Comparative Study of Reactive and Predictive Scaling Approaches for Redis Clusters

Bachelorarbeit

zur Erlangung des akademischen Grades

„Bachelor of Science in Engineering“

eingereicht am Studiengang Wirtschaftsinformatik

Studienrichtung: Wirtschaftsinformatik

Verfasst von: Johann Krischan Hipolito

Betreuung: Michael Nestler

Graz, 2026

FH | JOANNEUM
University of Applied Sciences

Ehrenwörtliche Erklärung

Hier steht die ehrenwörtliche Erklärung.

Table of Contents

Ehrenwörtliche Erklärung i
Table of Contents ii
List of Figures iv
List of Tables v
List of Abbreviations vi
Kurzfassung vii
Abstract viii
1 Introduction 1
1.1 Background 1
1.2 Problem Statement 1
1.3 Objectives 1
2 Theoretical Foundations 2
2.1 Kubernetes Concepts Overview 2
2.1.1 Use Cases of Kubernetes 2
2.2 Autoscaling in Kubernetes 2
2.3 Stateful vs. Stateless Workloads 2
2.3.1 Problems with Stateful Workloads in Kubernetes 2
2.4 Metrics and Observability in Kubernetes 2
3 Related Works into Kubernetes Autoscaling with Stateful Workloads . . . 3
3.1 Existing Research into Stateful Kubernetes Autoscaling 3
3.2 Industry Implementations and Best Practices 3
3.3 Research Gaps in Current Research and Opportunities for Improvement 3
4 Methodology and Experimental Setup 4
4.1 Experimental Setup 4
4.1.1 Virtualisation Environment 4
4.1.2 Routing 4
4.1.3 Redis Cluster Configuration 6
4.1.4 FastAPI Application Design 6
4.2 Autoscaling Configuration 7
4.2.1 Initial Design and Configuration 7
4.2.2 Technical Constraint 8
4.2.3 Working Design 8
4.3 Load Simulation 9
4.3.1 Workload Distribution 9
4.3.2 Simulation Stages 10
4.3.3 Performance Thresholds 10
4.4 Evaluation Metrics and Data Collection 10
4.4.1 Compute and Memory Saturation 11
4.4.2 Network and Data Pipeline 11
4.4.3 Application Throughput and Reliability 11
5 Results and Analysis 13
5.1 Compute and Memory Saturation 13
5.2 Network and Data Pipeline 13
5.3 Application Throughput and Reliability 13

5.4 Final Comparison between Reactive Scaling and Predictive Scaling . . 13
6 Conclusion 14
6.1 Summary of Findings 14
6.2 Discussion 14
Appendix 15
Glossary 16
List of References 17
Attachments 18

List of Figures

1 High-level architecture overview of the experimental setup. 5

List of Tables

1 IP allocation and node roles within the Proxmox virtualisation environment. 6

List of Abbreviations

API Application Programming Interface

CRD Custom Resource Definition

HPA Horizontal Pod Autoscaler

JSON JavaScript Object Notation

KEDA Kubernetes Event-Driven Autoscaling

K3s Lightweight Kubernetes distribution by Rancher

K8s Kubernetes

yaml YAML Ain’t Markup Language

Kurzfassung

Die Migration zustandsbehafteter Workloads, wie etwa partitionierter Datenbanken wie Redis, nach Kubernetes verursacht erhebliche architektonische Konflikte. Native Kubernetes-Skalierungsmechanismen, insbesondere der Horizontal Pod Autoscaler (HPA) und der `StatefulSet`-Controller, gehen von der generellen Austauschbarkeit von Rechenressourcen aus. Bei Scale-in-Vorgängen verwenden sie einen topologieblinden Löschmechanismus in umgekehrter Reihenfolge, der notwendige Datenmigrationsprotokolle umgeht und einen katastrophalen Datenverlust riskiert.

Diese Arbeit untersucht diese architektonischen Einschränkungen empirisch anhand eines deterministischen, lastfreien A/B-Tests. Die Studie evaluiert den Scale-in eines Redis-Clusters von vier auf drei Master-Knoten mit 5000 Datensätzen (Keys). Dabei wird die native HPA-Baseline mit dem Opstree Redis Operator, gesteuert durch Kubernetes Event-driven Autoscaling (KEDA), verglichen.

Die empirischen Ergebnisse zeigen, dass ein nativer HPA-Scale-in einen massiven Datenverlust verursacht. Dies führte zum sofortigen Verlust von 25 % des Datensatzes (1250 Keys), 4096 verwaisten Hash-Slots und einer clusterweiten Nichtverfügbarkeit. Im Gegensatz dazu fing der Operator den Skalierungsbefehl erfolgreich ab, um einen präemptiven, topologiebewussten Resharding-Prozess zu orchestrieren. Dies ermöglichte einen reibungslosen Scale-in ohne Datenverlust und mit durchgehender Verfügbarkeit.

Letztendlich belegt diese Untersuchung, dass die Implementierung domänenspezifischer Orchestrierungslogik durch das Operator-Pattern nicht nur eine operative Erleichterung, sondern eine absolute architektonische Notwendigkeit darstellt, um die Datenintegrität bei der Skalierung zustandsbehafteter Workloads in Kubernetes zu gewährleisten.

Abstract

The migration of stateful workloads, such as sharded databases like Redis, into Kubernetes introduces significant architectural friction. Native Kubernetes scaling primitives, specifically the Horizontal Pod Autoscaler (HPA) and the `StatefulSet` controller, operate on the assumption of compute fungibility. During scale-in operations, they employ a topology-blind, reverse-ordinal pod deletion mechanism that bypasses necessary data migration protocols, risking catastrophic data loss.

This thesis empirically investigates these architectural limitations through a deterministic, quiescent A/B test. The study evaluates a scale-in operation from four to three master nodes on a Redis cluster containing 5000 keys. It compares the native HPA baseline against the Opstree Redis Operator, driven by Kubernetes Event-driven Autoscaling (KEDA).

The empirical findings demonstrate that a native HPA scale-in induces a severe data cliff. This resulted in the immediate loss of 25% of the dataset (1250 keys), 4096 orphaned hash slots, and cluster-wide unavailability. Conversely, the operator successfully intercepted the scaling command to orchestrate a pre-emptive, topology-aware resharding process. This achieved a graceful scale-in with zero data loss and continuous cluster availability.

Ultimately, this research proves that encoding domain-specific orchestration logic via the Operator pattern is not merely an operational convenience, but an absolute architectural necessity for safeguarding data integrity when scaling stateful workloads in Kubernetes.

1 Introduction

1.1 Background

1.2 Problem Statement

1.3 Objectives

2 Theoretical Foundations

2.1 Kubernetes Concepts Overview

2.1.1 Use Cases of Kubernetes

2.2 Autoscaling in Kubernetes

2.3 Stateful vs. Stateless Workloads

2.3.1 Problems with Stateful Workloads in Kubernetes

2.4 Metrics and Observability in Kubernetes

3 Related Works into Kubernetes Autoscaling with Stateful Workloads

3.1 Existing Research into Stateful Kubernetes Autoscaling

3.2 Industry Implementations and Best Practices

3.3 Research Gaps in Current Research and Opportunities for Improvement

4 Methodology and Experimental Setup

4.1 Experimental Setup

This section details the foundational infrastructure constructed to evaluate the selected autoscaling methodologies. It outlines the underlying virtualization environment, the internal network routing, and the deployment configurations for both the stateful Redis cluster and the FastAPI backend. Establishing this controlled architecture ensures a consistent and reproducible baseline for the subsequent load simulation and metric evaluation phases.

4.1.1 Virtualisation Environment

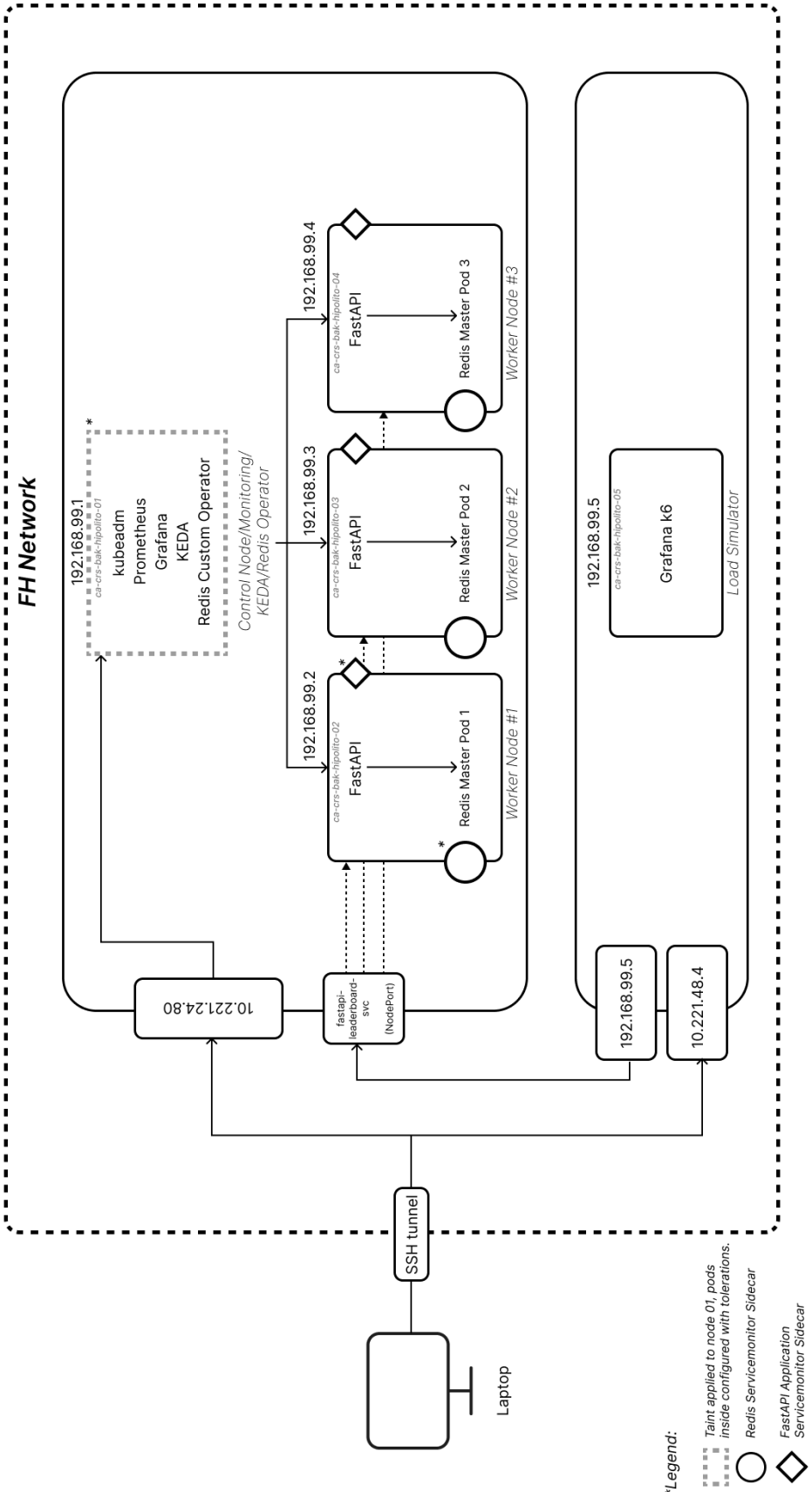
The experimental infrastructure was hosted within the FH Joanneum university cloud network, utilizing Proxmox as the underlying virtualization platform. To establish an isolated and reproducible testing environment, a dedicated pool of five Virtual Machines (VMs) was provisioned. As illustrated in the high-level architecture overview, these five VMs (ca-crs-bak-hipolito-01 through 05) are logically partitioned into three distinct operational roles:

- Control and Monitoring Node (1 VM): Provisioned at IP 192.168.99.1, this node manages the Kubernetes control plane via kubeadm and centralizes the infrastructure's management services, including Prometheus, Grafana, KEDA, and the Redis Custom Operator. To ensure workload stability and prevent resource contention with the primary applications, a taint is applied to this node; only administrative and monitoring pods configured with the appropriate tolerations are scheduled here.
- Worker Nodes (3 VMs): Provisioned at IPs 192.168.99.2, 192.168.99.3, and 192.168.99.4, these machines execute the core experimental workloads. They host the distributed FastAPI application instances and the stateful Redis Master pods required for the autoscaling evaluation.
- Load Simulation Node (1 VM): Operating at IP 192.168.99.5, this machine is strictly dedicated to generating simulated traffic using Grafana k6. Isolating the load simulator on a separate VM prevents the computational overhead of the test generation from skewing the performance metrics and CPU utilization of the worker nodes.

4.1.2 Routing

External access to the experimental environment within the FH Network is securely established from a local machine via an SSH tunnel. This tunnel bridges the external connection to the internal network infrastructure, which handles traffic direction to the designated virtual machines.

Figure 1: High-level architecture overview of the experimental setup.



The routing architecture employs a dual-network IP strategy to manage traffic flow:

- Administrative and Simulator Access: Intermediate network IPs map traffic to the internal virtual machine subnet. Specifically, the IP address 10.221.24.80 routes to the main control and monitoring node at 192.168.99.1, while 10.221.48.4 provides direct routing to the dedicated load simulator node at 192.168.99.5.
- Application Traffic Distribution: Traffic intended for the backend application is directed to a Kubernetes service named fastapi-leaderboard-svc. This service is explicitly configured as a NodePort, acting as the primary entry point for application requests and balancing the incoming traffic across the FastAPI pods residing on the three worker nodes.

Hostname	External IP Address(eth0)	Internal IP Address (eth1)
ca-crs-bak-hipolito-01	10.221.24.80	192.168.99.1/24
ca-crs-bak-hipolito-02	10.221.24.81	192.168.99.2/24
ca-crs-bak-hipolito-03	10.221.24.82	192.168.99.3/24
ca-crs-bak-hipolito-04	10.221.24.83	192.168.99.4/24
ca-crs-bak-hipolito-05	10.221.48.84	192.168.99.5/24

Table 1: IP allocation and node roles within the Proxmox virtualisation environment.

4.1.3 Redis Cluster Configuration

The stateful data tier of the experimental architecture consists of a distributed Redis cluster managed by a Redis Custom Operator, utilizing the Opstree Redis Operator. To maintain a strict separation of control and data planes, the operator itself is deployed on the main control and monitoring node (ca-crs-bak-hipolito-01).

The core database workloads are handled by three distinct Redis Master pods. These pods are explicitly distributed across the three available worker nodes (ca-crs-bak-hipolito-02, ca-crs-bak-hipolito-03, and ca-crs-bak-hipolito-04) to ensure high availability and replicate a robust stateful deployment topology.

For telemetry and observability, each Redis Master pod is provisioned with an attached Redis Servicemonitor sidecar. These sidecar containers expose real-time performance and resource utilization metrics, which are continuously scraped by the central Prometheus instance located on the control node. This metric aggregation pipeline establishes the foundational data source required for the autoscaling mechanisms evaluated in the subsequent sections.

4.1.4 FastAPI Application Design

The FastAPI application serves as the computational backend for the experimental setup, handling incoming simulated traffic routed through the `fastapi-leaderboard-svc` NodePort service. The application provides a RESTful interface designed to manage

distributed leaderboard operations, exposing three primary endpoints for the virtual users:

- `POST /player/{player_id}/score`: Processes write pressure by accepting a player ID and a new score payload. It routes the update to the appropriate database shard and utilizes the Redis `ZADD` command to update the player's standing.
- `GET /leaderboard`: Handles read workloads by retrieving the top N players globally. It accepts an optional query parameter to define the slice size and returns the highest-scoring players alongside their injected ranks.
- `GET /player/{player_id}/rank`: Retrieves the specific global rank of an individual player. If the player exists, it returns their exact score and calculates their global standing.

To manage data distribution across the Redis Master pods residing on the worker nodes (`ca-crs-bak-hipolito-02`, `ca-crs-bak-hipolito-03`, and `ca-crs-bak-hipolito-04`), the FastAPI backend implements application-level sharding coupled with an asynchronous Scatter-Gather architecture. The global leaderboard is logically partitioned into 12 distinct shards.

For write operations, the application employs a deterministic hashing function (`zlib.crc32`) against the UTF-8 encoded `player_id`. A modulo operation assigns the player consistently to one of the 12 shards, ensuring uniform data distribution.

For read operations requiring a global view, the application executes a Scatter-Gather pattern. It leverages Python's `asyncio.gather` to scatter parallel read requests—such as `ZREVRANGE` for fetching top players or `ZCOUNT` for calculating rank offsets—across all 12 shards simultaneously. Once the application collects all shard responses, it gathers the data in memory, flattens and sorts the arrays, and computes the final global state before returning the response payload. This design distributes the computational load of sorting and ranking across the entire stateful backend cluster.

4.2 Autoscaling Configuration

This section details the infrastructure engineered to execute the reactive and predictive autoscaling methodologies. It outlines the original architectural intent, the integration challenges encountered with the stateful data tier, and the final custom software developed to orchestrate automated scaling events.

4.2.1 Initial Design and Configuration

The preliminary architecture utilized Kubernetes Event-driven Autoscaling (KEDA) to manage the Redis cluster's capacity by deploying `ScaledObject` manifests targeting the Opstree `RedisCluster` Custom Resource Definition. KEDA was configured to poll

the in-cluster Prometheus service (`http://prometheus-operated.monitoring.svc.cluster.local:9090`) every 15 seconds to evaluate scaling rules based on HTTP request rates. To prevent metric evaluation errors during periods of absolute zero traffic, all PromQL queries were appended with the `or vector(0)` modifier.

Two distinct `ScaledObject` configurations were designed to evaluate the reactive and predictive paradigms:

- **Reactive Autoscaler:** This configuration defined a cluster size between 3 and 4 replicas. It utilized a direct PromQL query to calculate the immediate Requests Per Second (RPS) over a two-minute rolling window (`sum(rate(http_request_duration_seconds_count{namespace="fastapi-leaderboard"}[2m]))`). The scale-up trigger was set to a raw threshold of 65 RPS. To control scaling velocity, the underlying Horizontal Pod Autoscaler (HPA) was configured with a 0-second stabilization window for scaling up, limiting the addition to 1 pod every 180 seconds. For scaling down, it enforced a 300-second stabilization window, removing at most 1 pod every 120 seconds to prevent destructive cluster oscillation.
- **Predictive Autoscaler:** Designed to handle more dynamic load variations, this configuration expanded the maximum limit to 6 replicas. It employed the PromQL `predict_linear` function (`predict_linear(sum(rate(...[2m]))[5m:15s], 120)`) to analyze the HTTP request rate over the preceding 5 minutes, projecting anticipated traffic 120 seconds into the future. If the forecasted traffic breached the 65 RPS threshold, KEDA preemptively initiated a scale-up event. To maintain cluster stability and prevent the "thundering herd" problem—where multiple pods are provisioned simultaneously, leading to severe resource contention during state rebalancing—the HPA scale-up policy was strictly throttled to add only 1 pod every 60 seconds.

4.2.2 Technical Constraint

During the initial load testing phases, a fundamental architectural mismatch was identified between KEDA and the Opstree Redis Operator. KEDA natively interfaces with standard Kubernetes workload resources (such as Deployments or StatefulSets) or CRDs that expose a standard `/scale` subresource. However, scaling a distributed Redis cluster requires state-aware operations, including provisioning new Master pods and safely rebalancing the 16,384 logical hash slots across the new topology. KEDA lacked the native capability to orchestrate these complex, operator-specific patching sequences, rendering direct integration unviable without a dedicated middleware bridge.

4.2.3 Working Design

To bypass the KEDA integration constraints, a custom Python-based autoscaling daemon was developed to serve as a direct bridge between the cluster telemetry and the Opstree Redis Operator. Operating from the central control node, this daemon runs

as a continuous background process, managing the scaling lifecycle through a defined polling interval of 10 seconds.

The daemon retrieves real-time traffic metrics by querying the active Prometheus HTTP API. Specifically, it executes a PromQL query to calculate the aggregate Requests Per Second (RPS) processed by the FastAPI application over a two-minute rolling window (`sum(rate(http_request_duration_seconds_count{namespace="fastapi-leaderboard"}[2m]))`).

Scaling decisions are governed by a defined threshold of 65.0 RPS:

- **Scale-Up Event:** If the aggregated RPS breaches the 65.0 threshold, the daemon immediately triggers a scale-up event. It utilizes Python's `subprocess` module to execute a `kubectl patch` command, modifying the `clusterSize` specification of the `RedisCluster` Custom Resource Definition (CRD) via a merge patch. This increases the stateful replica count from the baseline of 3 to a maximum of 4 nodes.
- **Scale-Down Event (Stabilization Window):** To prevent destructive oscillation—or "flapping"—during transient traffic dips, the daemon implements a 300-second (5-minute) cooldown period. The incoming traffic must consistently remain below the 65.0 RPS threshold for the entire duration of this stabilization window before the daemon patches the CRD to scale the cluster back down to 3 replicas.

By programmatically patching the CRD directly, this daemon forces the Opstree Operator to acknowledge the updated `clusterSize`. This successfully delegates the complex tasks of pod provisioning and hash slot rebalancing to the operator while maintaining absolute control over the custom autoscaling logic.

4.3 Load Simulation

To systematically evaluate the autoscaling configurations, a comprehensive load testing suite was implemented using Grafana k6. The simulation is executed from the dedicated load simulator node (`ca-crs-bak-hipolito-05`)[cite: 1] to ensure test generation overhead does not interfere with the worker nodes processing the traffic. The k6 script targets the FastAPI NodePort service exposed at `http://ca-crs-bak-hipolito-01:31964`.

4.3.1 Workload Distribution

The simulation models realistic user behavior across a population pool of 5,000 distinct virtual players. Each Virtual User (VU) executes a continuous loop with a one-second pause between requests. To accurately reflect a read-heavy gaming leaderboard scenario, the script employs randomized branching to distribute traffic across the three API endpoints:

- **Write Operations (20%):** Submits new randomized scores via `POST /player/{id}/score`.

- **Rank Reads (70%):** Queries individual player standings via `GET /player/{id}/rank`.
- **Global Reads (10%):** Retrieves the top 100 players via `GET /leaderboard?top=100`.

4.3.2 Simulation Stages

The test executes a precise 31-minute load profile designed to simulate a full daily traffic cycle, exposing both auto scaling paradigms to various rate-of-change scenarios:

- **Night Baseline (0:00 – 5:00):** Holds steady at 10 VUs. This establishes a baseline and allows the predictive algorithm's 5-minute lookback window to initialize with valid telemetry data.
- **Morning Ramp (5–9 min):** A gradual climb to 40 VUs to evaluate how effectively the linear forecasting anticipates a steady upward trend.
- **Lunch Peak (9–17 min):** Sustained heavy pressure at 80 VUs. This forces the Opstree Operator to execute pod provisioning and data re-sharding under load.
- **Afternoon Plateau (17–21 min):** A step down to 60 VUs to measure post-scale stabilization and resource utilization.
- **Flash Spike (21–22 min):** An aggressive 1-minute burst to 120 VUs, simulating sudden application virality.
- **Premature Drop (22 min):** A sudden 10-second cliff dropping from 120 down to 20 VUs. This specifically tests predictive resource waste, evaluating whether the predictive scaler provisions unnecessary pods right as the traffic vanishes.
- **Cooldown (22–31 min):** A sustained low-traffic period at 20 VUs (tapering to 0) to observe idle pod allocation and the execution of the 5-minute scale-down stabilization window.

4.3.3 Performance Thresholds

To automatically quantify system degradation, Service Level Agreement (SLA) thresholds are integrated directly into the k6 script. Latency is tracked per endpoint across the 50th, 95th, and 99th percentiles (p50, p95, p99). For instance, the computationally expensive global leaderboard read requires a p99 latency below 1500 ms, whereas individual rank reads require a p99 below 600 ms. If the autoscaling mechanisms fail to provision resources fast enough, the resulting latency spikes will exceed these defined thresholds, providing concrete failure points for evaluation.

4.4 Evaluation Metrics and Data Collection

To systematically compare the reactive and predictive autoscaling methodologies, performance data is collected across three primary infrastructural domains. The evaluation

focuses on resource saturation, network pipeline efficiency, and overall application reliability during dynamic scaling events.

4.4.1 Compute and Memory Saturation

The first category evaluates the computational overhead induced by the autoscaling mechanisms themselves, particularly during stateful data operations:

- **Re-sharding CPU Penalty:** Tracks the exact CPU utilization spikes on existing Redis worker nodes when the Opstree Operator forces them to synchronize data and migrate logical hash slots to a newly provisioned pod.
- **Memory Pressure During Scaling:** Monitors node-level memory consumption via Prometheus to evaluate if the state synchronization process risks triggering Out-Of-Memory (OOM) kills on worker nodes that are simultaneously processing peak request volumes.
- **Control Plane Overhead:** Measures the CPU and memory consumption of the k3s control plane, the autoscaling components (KEDA and the custom daemon), and the Opstree Operator during the exact moments of scaling and forecasting. This isolates the orchestrator's infrastructural footprint from the primary workload.

4.4.2 Network and Data Pipeline

The second category analyzes the impact of distributed scaling on the underlying virtualized network:

- **Internal Bandwidth Consumption:** Measures the network I/O spikes between the campus-hosted virtual machines while the Redis cluster rebalances its state over the internal Proxmox network.
- **Data Synchronization Duration:** Quantifies the exact time window starting from when a new Redis pod is scheduled by the Kubernetes control plane to when it is fully synchronized and actively accepting routed TCP connections from the FastAPI backend.

4.4.3 Application Throughput and Reliability

The final category assesses the direct impact of the autoscaling paradigms on the end-user experience, utilizing the telemetry generated by the Grafana k6 load simulator:

- **Throughput Degradation:** Tracks the total successful Requests Per Second (RPS) alongside sustained latency percentiles (p50 and p99) over the entire duration of the simulated traffic cycle.

- **Client Disturbance:** Measures acute client-side latency degradation—specifically isolating the 95th (p95) and 99th (p99) percentiles—during the resource-intensive cluster re-sharding phases.
- **Predictive Resource Waste:** Evaluates the algorithmic accuracy of the predictive forecasting model. This metric assesses how frequently the predictive algorithm provisions a new Redis pod that sits idle because the forecasted load spike ended prematurely or failed to materialize.

5 Results and Analysis

5.1 Compute and Memory Saturation

5.2 Network and Data Pipeline

5.3 Application Throughput and Reliability

5.4 Final Comparison between Reactive Scaling and Predictive Scaling

6 Conclusion

6.1 Summary of Findings

6.2 Discussion

Appendix

Glossar

ConfigEpoch A collision-resolution concept in a Redis Cluster used to determine which node holds the authoritative claim over a specific hash slot during split-brain or network partition events.

CRD (Custom Resource Definition) An API extension mechanism in Kubernetes that allows users to define and instantiate custom resources, such as the `RedisCluster` resource utilized by the Opstree operator.

Data Cliff A term utilized in this thesis to describe the sudden, catastrophic loss of data and cluster availability that occurs when a stateful node is terminated without prior data migration.

Ghost Pod A pod provisioned by a generic autoscaler (like HPA) that joins a Kubernetes `StatefulSet` but remains unintegrated into the database topology, meaning it holds zero hash slots and serves no traffic.

Hash Slot The fundamental unit of data partitioning in a Redis Cluster. The total keyspace is divided into exactly $16\,384$ hash slots, which must be distributed across the master nodes to maintain cluster availability.

Horizontal Pod Autoscaler (HPA) A native Kubernetes control loop that automatically updates a workload resource (such as a `StatefulSet`) to match demand based on observed metrics, such as average CPU utilization.

KEDA (Kubernetes Event-driven Autoscaling) A single-purpose, lightweight component added to a Kubernetes cluster to drive autoscaling based on external event metrics, such as Prometheus cache-miss ratios.

Operator Pattern A software extension to Kubernetes that encodes domain-specific operational knowledge into a custom controller to automate the lifecycle and topology management of complex stateful applications.

Quiescent A state of execution where no external read or write traffic is applied to the database. This state is used in the experimental methodology to isolate scaling mechanics from load-induced resharding interruptions.

Resharding The process of live-migrating hash slots, along with the underlying data they contain, from one node to another within an active database cluster.

Stateful Workload An application or database (such as Redis) that persists client data across sessions, meaning its underlying compute pods contain unique data and are not strictly interchangeable.

StatefulSet A native Kubernetes workload API object used to manage stateful applications, providing strict guarantees about the ordering and uniqueness of a set of Pods, including the enforcement of reverse-ordinal deletion during scale-in operations.

List of References

Attachments

The following Attachments are included as a digital supplement to this thesis (as a ZIP archive in the submission system) and reference the complete codebase used for conducting the experiments.

Attachment 1: Project repository (Quellcode)
: The complete GitHub repository contains all Kubernetes manifests, Bash automation scripts (`data-safety-experiment.sh`), FastAPI deployment files, and the raw Markdown output logs of the conducted experiments.
URL: https://github.com/gitbudy360/bac-johann-hipolito
Commit-Hash: `be45110ff64e3f3a5527c91c6eba6a5c7270d18a`

Attachment 2: Grafana Dashboards
: The JavaScript Object Notation (JSON)-exports of the Grafana dashboards used for visualizing hash slot assignments and key metrics during the scale-in phases. Located in the `/k8s-cluster/grafana/` directory.